



demo5-data-jpa: Files required

1. MyBootApplication.java	Updated in Demo5
2. Customer.java	Same as Demo4
3. CustomerDAO.java	Updated in Demo5
4. application.properties	Same as Demo4
5. pom.xml	Same as Demo4
6. mycustomers table	Same as Demo4

1) MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.data.domain.Sort.Order;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerDAO custDAO;

    @Override
    public void run(String... args) throws Exception {

        List<Customer> mylist=null;

        //1. getAllCustomers - phoneSort
        Sort phoneSort=Sort.by(Order.asc("phone"));
        mylist=custDAO. getAllCustomers(phoneSort);
        mylist.forEach(System.out::println);
    }
}
```



```
//1. getAllCustomers - nameSort
Sort nameSort=Sort.by(Order.asc("cname"));
mylist=custDAO. getAllCustomers(nameSort);
mylist.forEach(System.out::println);

//2. getCustomersByCity - phoneSort
mylist=custDAO. getCustomersByCity("Blore",phoneSort);
mylist.forEach(System.out::println);

//2. getCustomersByCity - nameSort
mylist=custDAO. getCustomersByCity("Blore",nameSort);
mylist.forEach(System.out::println);

//3. getAllCustomers - mypage1
Pageable mypage1=PageRequest.of(0, 5);
mylist=custDAO. getAllCustomers(mypage1);
mylist.forEach(System.out::println);

//3. getAllCustomers - mypage2
Pageable mypage2=PageRequest.of(0, 5,Sort.by(Order.asc("cname")));
mylist=custDAO. getAllCustomers(mypage2);
mylist.forEach(System.out::println);

//4. getCustomersByCity - mypage3
Pageable mypage3=PageRequest.of(0, 3);
mylist=custDAO.getCustomersByCity("Blore",mypage3);
mylist.forEach(System.out::println);

//4. getCustomersByCity - mypage4
Pageable mypage4=PageRequest.of(0, 3,Sort.by(Order.asc("phone")));
mylist=custDAO. getCustomersByCity("Blore",mypage4);
mylist.forEach(System.out::println);

System.out.println("Done!!!");
}

public static void main(String[] args) {
SpringApplication.run(MyBootApplication.class, args);
}

}
```



3) CustomerDAO.java

```
package com.jlcindia.springboot;

import java.util.List;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public interface CustomerDAO extends JpaRepository<Customer, Integer> {

    @Query("from Customer cust ")
    List<Customer> getAllCustomers(Sort sort);

    @Query("from Customer cust where city=?1")
    List<Customer> getCustomersByCity(String city,Sort sort);

    @Query("from Customer cust")
    List<Customer> getAllCustomers(Pageable pageable);

    @Query("from Customer cust where city=?1")
    List<Customer> getCustomersByCity(String city,Pageable pageable);

}
```



Spring DATA Mongo

- ♦ CRUD Operations called Insert, Update, Retrive and Delete Opearions are common across all DAOs. The only difference is domain types you are interacting with.
- ♦ Spring Data internally provides the implementation for all such common methods. So developer is not responsible for writing implementation for these common methods.

Ex:

```
public interface CustomerDAO extends MongoRepository<Customer,  
String>{  
    //Empty  
}
```

- ♦ **MongoRepository** is parameterized and takes two parameters
 - Domain class
 - Object Id Type
- ♦ **CrudRepository** is the root for All the Dpring Data Repositories.
- ♦ CrudRepository has 12 Methods.
- ♦ **PagingAndSortingRepository** extends CrudRepository
- ♦ PagingAndSortingRepository has 2 Methods.
- ♦ **MongoRepository** extends PagingAndSortingRepository.
- ♦ **MongoRepository** has 2 methods extra and 14 methods will be inherited.
- ♦ Your DAO interface which is extending **MongoRepository** can use all these methods for performing common persistence operations such as saving, deleting, finding

Methods of PagingAndSortingRepository

1.	Page<T> findAll(Pageable pageable)
2.	Iterable<T> findAll(Sort sort)



Methods of CrudRepository

1.	long count()
2.	void delete(T entity)
3.	void deleteAll()
4.	void deleteAll(Iterable<? extends T> entities)
5.	void deleteAllById(Iterable<? extends ID> ids)
6.	void deleteById(ID id)
7.	boolean existsById(ID id)
8.	Iterable<T> findAll()
9.	Iterable<T> findAllById(Iterable<ID> ids)
10.	Optional<T> findById(ID id)
11.	S save(S entity)
12.	Iterable<S> saveAll(Iterable<S> entities)

Methods of JpaRepository

1.	S insert(S entity)
2.	List<S> insert(Iterable<S> entities)

demo6-data-mongo: Working Steps

A) Setup MongoDB

- 1) Make sure that MongoDB is installed in your Machine
- 2) Open MongoDB Compass and Connect with the following URL:

mongodb://localhost:27017/

- 3) Create the Database called **myjlcdb5**
- 4) Create the Collection called **mycustomers**
- 5) Insert some **5 Documents** in **mycustomers** Collection

```
[ {  
  "cid": 101,  
  "cname": "Sri",  
  "email": "sri@jlc",  
  "phone": 111,  
  "city": "Blore",  
  "balance": 15000  
},
```



```
{
  "cid": 102,
  "cname": "Hello",
  "email": "hello@jlc",
  "phone": 333,
  "city": "Blore",
  "balance": 25000
},
{
  "cid": 103,
  "cname": "SD",
  "email": "sd@jlc",
  "phone": 444,
  "city": "Hyd",
  "balance": 18000
},
{
  "cid": 104,
  "cname": "DS",
  "email": "ds@jlc",
  "phone": 555,
  "city": "Hyd",
  "balance": 8000
},
{
  "cid": 105,
  "cname": "Hai",
  "email": "hai@jlc",
  "phone": 222,
  "city": "Blore",
  "balance": 12000
}
]
```

B) Create Spring Boot Project

6) Copy demo1-data-jpa project and paste as demo6-data-mongo

7) Update **pom.xml**

```
<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>
</dependencies>
```



8) Update **application.properties**

```
# MongoDB Properties
#spring.data.mongodb.authentication-database=admin
#spring.data.mongodb.username=srinivas
#spring.data.mongodb.password=srinivas
spring.data.mongodb.database=myjlcdb5
spring.data.mongodb.port=27017
spring.data.mongodb.host=localhost

#Log Properties
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %m %n
```

9) Update **Customer.java**

```
@Document(collection = "mycustomers")
public class Customer {

    @Id
    private String myobjectId;

    private String cid;
    private String cname;
    private String email;
    private int phone;
    private String city;
    private double balance;

    //Constructors
    //Setters and Getters
    //toString()

}
```

10)Update **CustomerDAO.java**

```
@Repository
public interface CustomerDAO extends MongoRepository<Customer, String> {

}
```

11)Update **MyBootApplication.java**

⇒ You can call and test methods from **MongoRepository**



demo6-data-mongo: Files required

1. MyBootApplication.java	
2. Customer.java	
3. CustomerDAO.java	
4. application.properties	
5. pom.xml	

1) MyBootApplication.java

```
package com.jlcindia.springboot;

import java.util.List;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.data.domain.Sort;
import org.springframework.data.domain.Sort.Order;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerDAO custDAO;

    @Override
    public void run(String... args) throws Exception {

        List<Customer> mylist=null;

        // 1.save() method - inserting
        Customer cust1 = new Customer("106", "vas", "vas@jlc", 12345, "Blore", 7500);
        custDAO.save(cust1);

        // 2.insert() method - inserting
        Customer cust2 = new Customer("108","ppp", "ppp@jlc", 8888, "Blore",12500);
        custDAO.insert(cust2);
```



// 3.save() method - Updating

```
Optional<Customer> myOps1 = custDAO.findById("611d0d8234971e6cc53f6791");
if(myOps1.isPresent()) {
    Customer cust = myOps1.get();
    cust.setCid("107");
    cust.setCname("ddd");
    cust.setEmail("ddd@jlc");
    cust.setPhone(55555);
    custDAO.save(cust);
}
```

//4. existsById

```
boolean b= custDAO.existsById("611cd83bd22f1e5adfbf6a95");
System.out.println("Document Found : "+ b);
```

//5. findById

```
Optional<Customer> myOps2 = custDAO.findById("611cd83bd22f1e5adfbf6a95");
myOps2.ifPresent(System.out::println);
```

//6. deleteById

```
custDAO.deleteById("611d00e1cd9ff57d738acc08");
```

//7. delete

```
Optional<Customer> myOps3 = custDAO.findById("611d02cd739d2c69846025e8");
myOps3.ifPresent(mycust -> custDAO.delete(mycust));
```

//8. count

```
System.out.println("Total Customer : "+ custDAO.count());
```

//9. findAll()

```
mylist=custDAO.findAll();
mylist.forEach(System.out::println);
```

//10. findAll(Sort)

```
Sort nameSort = Sort.by(Order.asc("cname"));
mylist = custDAO.findAll(nameSort);
mylist.stream().forEach(System.out::println);
```

//You can test the remaining methods

```
System.out.println("Done!!!");
}
public static void main(String[] args) {
    SpringApplication.run(MyBootApplication.class, args);
}
}
```



2) Customer.java

```
package com.jlcindia.springboot;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.mongodb.core.index.Indexed;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

@Document(collection = "mycustomers")
public class Customer {

    @Id
    private String myobjectId;

    @Indexed
    private String cid;

    @Indexed
    private String cname;
    private String email;
    private int phone;
    private String city;
    private double balance;

    public Customer() {}
    public Customer(String cid, String cname, String email, int phone, String city,double balance)
    {
        this.cid = cid;
        this.cname = cname;
        this.email = email;
        this.phone = phone;
        this.city = city;
        this.balance=balance;
    }

    //Setters and Getters
    //toString()

}
```



3) CustomerDAO.java

```
package com.jlcindia.springboot;

import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
@Repository
public interface CustomerDAO extends MongoRepository<Customer, String> {

}
```

4)application.properties

MongoDB Properties

```
#spring.data.mongodb.authentication-database=admin
#spring.data.mongodb.username=srinivas
#spring.data.mongodb.password=srinivas
spring.data.mongodb.database=myjlcdb5
spring.data.mongodb.port=27017
spring.data.mongodb.host=localhost
```

#Log Properties

```
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %m %n
```

5)pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.springboot</groupId>
<artifactId>demo6-data-mongo</artifactId>
<version>1.0</version>
<name>demo6-data-mongo</name>
<description>This is Spring Boot- MongoDB Example</description>
<url>https://www.jlcindia.com</url>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
<relativePath/>
</parent>
```



```
<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>

</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```